

The DevSecOps Field Book

A Complete, Slow, Honest Path from Linux Basics
to Senior Platform Engineer

James Clements III

September 2026

Preface: Why This Book Exists

You are moving toward a Senior DevSecOps / Platform Engineer role. That role asks for a specific, wide stack of skills: Linux and scripting fluency, cloud operations on AWS and Azure, Infrastructure as Code with Terraform, container orchestration with Kubernetes, CI/CD pipeline design, security scanning embedded into every stage of the pipeline, Windows Active Directory and endpoint management, and the judgment to know how all of these pieces fit together into one coherent platform.

Most people who try to learn this stack make the same mistake: they learn each piece in isolation. They take an AWS course here, a Docker course there, a Terraform tutorial somewhere else, and they never combine them. They can recite what a VPC is and what a Kubernetes Pod is, but they have never watched a single commit travel from their laptop, through a security-scanning pipeline, into a built container image, into a running Kubernetes Deployment, on infrastructure that Terraform provisioned. That end-to-end motion — not the vocabulary — is what separates a candidate who can pass a trivia-style interview from an engineer a platform team actually wants to hire.

This book is built to avoid that trap. It follows a road map with five phases, in a deliberate order, because each phase is the foundation the next one stands on:

1. **Foundations** — Linux, the shell, and Git. Skipping this and jumping to "DevOps tools" is the single most common reason people stay stuck for years instead of months.
2. **Cloud basics** — not "AWS trivia," but the five or six concepts (compute, storage, networking, identity) that are the same on every cloud, learned on AWS because it has the largest market share and therefore the highest odds of matching a real job description.
3. **Infrastructure as Code** — Terraform, and more importantly the concept of describing infrastructure as reviewable, repeatable, version-controlled text instead of manual console clicks.
4. **Containers and orchestration** — Docker and Kubernetes, learned to the 20% depth that covers 80% of real production use, not chased to expert-level trivia.
5. **CI/CD** — the backbone of DevOps. Pipelines that actually move a real commit to a real running system, with security scanning wired directly into every stage, not bolted on afterward.

On top of that five-phase spine, this book adds the two things a *DevSecOps* specifically (rather than plain DevOps) role asks for: observability (Prometheus and Grafana, so you can see what your system is actually doing), and security as a first-class citizen at every phase rather than a separate audit at the end — secrets detection, static analysis, dependency and container scanning, Infrastructure-as-Code scanning, CIS hardening baselines, and the enterprise Windows/Active Directory layer that a platform engineer is expected to operate alongside all of the cloud-native tooling.

Every phase in this book is illustrated with a real, working case study: a small platform that was built from nothing, inside a git repository called **poetic-musings**, over the course of a few working sessions, specifically so you would have real code to read side by side with the explanation — not just theory. Where something genuinely could not be built in that environment (a real Windows Active Directory domain, a live Kubernetes cluster, a live cloud account), this book says so plainly, in a labeled “Honest Gap” box, and tells you exactly how to close it yourself, cheaply.

Read this book slowly. It is meant to be read once for the mental model, and then kept next to you as a reference while you build. There is no shortcut past building things yourself — but there is a shortcut past confusion, and that shortcut is reading things in the right order, with the right amount of explanation, the first time. That is what this book tries to be.

Contents

Preface: Why This Book Exists	iii
I Foundations	1
1 How to Use This Road Map	3
1.1 The five-phase sequence, and why order matters	3
1.2 The mistake almost everyone makes: learning in isolation	3
1.3 Personalizing the road map	4
2 Linux Fundamentals	5
2.1 Why Linux is the actual foundation	5
2.2 The filesystem and permissions	5
2.3 Processes and services	6
2.4 Networking basics at the OS level	6
2.5 Package management	7
3 Bash Scripting	9
3.1 Why scripting, not just typing commands	9
3.2 A first real script	9
3.3 Variables, conditionals, and loops	10
3.4 Functions and reusability	10
3.5 Command substitution and pipelines	10
4 Git	11
4.1 Why version control is non-negotiable	11
4.2 The core mental model	11
4.3 Branching and merging	11
4.4 Working with remotes (GitHub, GitLab)	12
4.5 The security-relevant Git habits	12
II Cloud Basics	15
5 Cloud Computing Concepts	17
5.1 Concepts first, provider second	17
5.2 The five concepts that matter	17

5.3	The five AWS services that cover 80% of real use	18
5.4	IAM: the concept that will follow you through the whole book	18
5.5	Certifications: after proof, not instead of it	19
III	Infrastructure as Code	21
6	Terraform and Infrastructure as Code	23
6.1	The problem IaC solves	23
6.2	State: Terraform’s memory	23
6.3	Providers, resources, and modules	24
6.4	The Terraform workflow	24
6.5	Hardening IaC by default	25
6.6	Scanning infrastructure code for security issues	25
IV	Containers and Orchestration	27
7	Docker and Containers	29
7.1	The problem containers solve	29
7.2	Images, containers, and the Dockerfile	29
7.3	Why every line above is a deliberate hardening choice	30
8	Kubernetes	33
8.1	The problem Kubernetes solves	33
8.2	The core objects	33
8.3	Liveness vs. readiness	33
8.4	The restricted Pod Security Standard, explained line by line	34
8.5	Network policy: default-deny	35
8.6	Kustomize: one base, many environments	35
V	CI/CD and Security Automation	37
9	CI/CD: The Backbone of DevOps	39
9.1	Why this is the highest-leverage phase	39
9.2	The anatomy of a pipeline	39
10	Shifting Security Left	41
10.1	What “shift left” actually means	41
10.2	The layers of automated security scanning	41
10.2.1	Secrets detection	41
10.2.2	SAST – Static Application Security Testing	41
10.2.3	SBOM and dependency scanning (SCA)	42
10.2.4	Container image scanning	42
10.2.5	IaC and Kubernetes manifest scanning	42
10.3	Pipeline design habits that mark seniority	42

VI	Observability	45
11	Prometheus and Grafana	47
11.1	Why observability is the natural last step	47
11.2	Metrics, the core concept	47
11.3	Alerting: the point of all of it	48
VII	The Enterprise Layer: Windows, Identity, and Hardening at Scale	49
12	Active Directory, GPOs, and Endpoint Management	51
12.1	Why this belongs in a cloud-native book	51
12.2	Active Directory: the identity backbone	51
12.3	Group Policy Objects: configuration as (a kind of) code	52
12.4	MECM/SCCM and Intune: the enforcement arm	52
13	CIS Benchmarks and Formal Hardening	53
13.1	What a CIS Benchmark actually is	53
13.2	From "aligned with" to "scored against"	53
14	Managed Development Environments: Coder	55
VIII	Putting It All Together	57
15	Avoiding the Isolated-Learning Trap, for Real	59
15.1	The mistake, restated	59
15.2	The discipline that fixes it	59
16	Consolidated Gap List and Next Exercises	61
16.1	Every honest gap named in this book, in one place	61
16.2	A study cadence	61
	Glossary	63
	About This Book	65

Part I

Foundations

Chapter 1

How to Use This Road Map

1.1 The five-phase sequence, and why order matters

Before any technical content, understand the shape of the whole journey, because the shape itself is the most important lesson in this book.

Phase	What you learn	Typical time
1. Foundations	Linux, Bash scripting, Git	1–2 months
2. Cloud basics	Compute, storage, networking, identity (on AWS)	1–2 months
3. Infrastructure as Code	Terraform concepts and provisioning	~1 month
4. Containers & orchestration	Docker, Kubernetes (the 20% that covers 80%)	1–2 months
5. CI/CD	Automated pipelines, end to end, with security gates	1–2 months
(<i>overlay</i>) Observability	Prometheus, Grafana	~1 month

Add that up and you get a genuinely achievable timeline: a few months of focused, sequenced effort, not years, and not a vague, sprawling “learn everything at once” approach. The key discipline is: **do not skip phase one to get to the exciting parts.** Everything in phases two through five assumes you are already comfortable at a Linux terminal, that you can write a Bash script without panicking, and that you understand Git well enough that branching and merging do not feel like magic. If those things are shaky, every later phase takes longer and hurts more, because you are simultaneously learning the new tool *and* the terminal skills you should already have.

1.2 The mistake almost everyone makes: learning in isolation

There is a specific failure mode worth naming clearly, because naming it is most of the cure. People take a cloud course, then a Docker course, then a Terraform course, then a CI/CD course — each one in its own bubble, each one ending with a short quiz or a toy exercise that never touches the other tools. They can define what a container is. They cannot explain what happens, step by step, when a developer pushes a commit and, twelve minutes later, a new version of the application is serving real traffic. They have a vocabulary. They do not have the connective tissue.

That connective tissue — the actual sequence of automated systems and hand-offs between them — is what makes an engineer valuable, because anyone can finish a Udemy course. Almost nobody,

without deliberate practice, has actually built and watched an end-to-end pipeline with their own hands. The fix is simple to state and takes real effort to do: **every time you learn a new tool, immediately combine it with everything you already know, on one continuously-growing project, instead of starting a new toy project from zero for every new tool.**

This book follows that discipline itself. Every phase below builds onto the *same* project, so that by the end of Part V you are not looking at five unrelated toy examples — you are looking at one small platform where the Terraform code provisions the registry the Docker image gets pushed to, which the Kubernetes manifests deploy, which the CI/CD pipeline wires together and scans for security issues at every step.

1.3 Personalizing the road map

If you are coming into this with prior experience — as a software engineer, a QA/test automation engineer, a sysadmin, a network engineer, or a database administrator — you are not starting from zero. A software engineer already has Git instincts and scripting comfort; spend relatively less time in Phase 1 and more time in cloud and infrastructure concepts that are genuinely new. A sysadmin already understands servers, networking, and operational discipline; Phase 1 and much of Phase 2 will feel familiar, and the genuinely new territory is Infrastructure as Code, containers, and CI/CD automation. Use your existing strengths as a running start, but do not skip the phases you are weak in just because you are strong somewhere else — a platform engineer is expected to be credible end to end.

Chapter 2

Linux Fundamentals

2.1 Why Linux is the actual foundation

Nearly every server, every container, every Kubernetes node, and every CI/CD runner you will ever touch in this career is running Linux. Not "usually." Essentially always. Learning DevOps without being fluent at a Linux terminal is like learning to drive without knowing how to work a clutch in a manual-transmission country — you can memorize directions to specific places, but you cannot actually operate the vehicle when something unexpected happens, and something unexpected *always* happens in production.

2.2 The filesystem and permissions

Linux organizes everything — files, directories, devices, even running processes — as part of one single tree rooted at /. Unlike Windows, there is no “C: drive” or “D: drive” concept; everything, including a second hard disk, gets *mounted* at some point inside that one tree.

```
/           # the root of everything
/etc        # system-wide configuration files
/var        # variable data: logs, caches, spool files
/var/log    # where most services write their logs
/home/<user> # a user's personal files
/usr/bin    # most installed programs live here
/tmp        # scratch space, often wiped on reboot
```

Every file and directory has an **owner** (a user), a **group**, and a set of **permissions** for three categories of people: the owner, the group, and everyone else. Permissions are read (r), write (w), and execute (x).

```
$ ls -l app.py
-rwxr-xr-- 1 jclements devteam 1420 Sep 19 10:02 app.py
# owner (jclements): read, write, execute
# group (devteam):  read, execute
# other:           read only
```

Why this matters for security

Every hardening choice you will read about later in this book — a container running as a non-root user, a file that should not be world-writable, a private key that must not be readable by anyone but its owner — is an application of exactly this permission model. If you deeply understand `rw`x and ownership here, every later “security best practice” will make immediate, mechanical sense instead of feeling like an arbitrary rule to memorize.

2.3 Processes and services

A running program is a **process**. Linux gives every process a Process ID (PID), tracks its resource usage, and lets you inspect, signal, and kill it.

```
ps aux                # list all running processes
top                  # live, updating view of process activity
kill -15 <pid>       # ask a process to terminate gracefully (SIGTERM)
kill -9 <pid>        # force-terminate immediately (SIGKILL), last resort
systemctl status nginx # check the status of a service managed by systemd
systemctl restart nginx # restart a service
journalctl -u nginx -f # follow a service's logs live
```

systemd is the init system used by virtually every modern Linux distribution — it is the first process that starts when the machine boots, and it is responsible for starting, stopping, and supervising every other background service (a “daemon”). When you later read a Kubernetes liveness probe restarting a crashed container automatically, recognize that as the same idea as systemd restarting a crashed service — “keep this thing running, and if it dies, bring it back” is a pattern that repeats at every layer of this stack.

2.4 Networking basics at the OS level

```
ip addr show          # show this machine's network interfaces and IPs
ss -tulpn             # show listening ports and which process owns each one
curl -v https://example.com # make an HTTP request, verbosely, to debug connectivity
ping 8.8.8.8          # test basic reachability
traceroute example.com # show the network hops to a destination
dig example.com        # query DNS for a hostname's IP address
```

When something in Kubernetes or the cloud “isn’t reachable,” the first fifteen minutes of debugging almost always happen at this level: is the process even listening on the port you expect (`ss -tulpn`), can you reach it at all (`curl`, `ping`), and does the name resolve to the address you think it does (`dig`).

2.5 Package management

Linux distributions differ mainly in how they install software. Debian/Ubuntu uses **apt**; Red Hat/CentOS/Amazon Linux uses **yum** or **dnf**.

```
sudo apt update && sudo apt install -y curl git      # Debian/Ubuntu
sudo dnf install -y curl git                          # Fedora / newer
RHEL
```

A DevSecOps-specific habit

Every package you install is code you did not write, running with real permissions on your system. Minimizing installed packages in a production image or server — installing only what is strictly required — is not laziness, it is a security control: fewer packages means a smaller *attack surface*, meaning fewer places a vulnerability could be hiding. You will see this exact instinct again in Chapter 7 when we build a minimal container image.

Chapter 3

Bash Scripting

3.1 Why scripting, not just typing commands

The moment you find yourself typing the same three or four commands, in the same order, more than twice, that sequence should become a script. Scripts are how you turn manual, error-prone, forgettable procedures into repeatable, reviewable, shareable automation — which is the entire spirit of DevOps applied at the smallest possible scale.

3.2 A first real script

```
#!/usr/bin/env bash
set -euo pipefail
# -e : exit immediately if any command fails
# -u : treat unset variables as an error, not a silent empty string
# -o pipefail : a pipeline fails if ANY command in it fails, not just
    the last one

LOG_DIR="/var/log/myapp"
RETENTION_DAYS=14

if [ ! -d "$LOG_DIR" ]; then
    echo "Log directory $LOG_DIR does not exist" >&2
    exit 1
fi

find "$LOG_DIR" -type f -mtime +"$RETENTION_DAYS" -name '*.log' -print -
delete

echo "Cleaned logs older than $RETENTION_DAYS days from $LOG_DIR"
```

`set -euo pipefail` is not optional

This single line is the difference between a script that fails loudly and immediately when something goes wrong, and a script that silently continues after a failure and does something worse three steps later. Get in the habit of starting every script with it. This is a small thing that

experienced engineers do automatically and beginners routinely skip, and it is one of the fastest tells in a code review of who has been burned by its absence before.

3.3 Variables, conditionals, and loops

```
NAME="poetic-musings"
COUNT=3

if [ "$COUNT" -gt 0 ]; then
    echo "$NAME has $COUNT pending items"
fi

for f in *.yaml; do
    echo "Checking $f"
done

while read -r line; do
    echo "Line: $line"
done < input.txt
```

3.4 Functions and reusability

```
log() {
    local level="$1"; shift
    echo "[$(date -u +%Y-%m-%dT%H:%M:%SZ)] [$level] $*"
}

log INFO "Starting deployment"
log ERROR "Something went wrong"
```

3.5 Command substitution and pipelines

```
FILE_COUNT=$(find . -name '*.py' | wc -l)
echo "Found $FILE_COUNT Python files"

grep -r "TODO" --include="*.py" . | sort | uniq -c | sort -rn
```

Where this shows up later

Almost every CI/CD pipeline step you will write in Part V is, underneath the YAML, a Bash script running on a CI runner. The YAML just decides *when* to run it and what environment to run it in. If Bash feels comfortable, CI/CD pipeline authoring will feel like writing scripts with slightly different plumbing, not like learning something entirely new.

Chapter 4

Git

4.1 Why version control is non-negotiable

Git tracks every change ever made to a set of files, who made it, when, and why (via a commit message), and lets many people work on the same codebase concurrently without overwriting each other's work. In DevOps and DevSecOps specifically, Git is not just where application code lives — it is also where Infrastructure as Code, Kubernetes manifests, and CI/CD pipeline definitions live, which means *every infrastructure change also gets a full history, a diff, and a code review*, exactly like an application change. This is one of the central ideas of the whole book and it starts here.

4.2 The core mental model

- A **repository** is a directory whose history Git is tracking.
- A **commit** is a saved snapshot of the whole repository at one point in time, with a unique hash identifying it, a message describing it, and a pointer to the commit(s) that came before it.
- A **branch** is just a movable pointer to a commit — creating a branch is nearly free, which is why Git branching is so central to how teams work in parallel.
- The **working tree** is the files on disk right now; the **staging area** (or “index”) is what you have told Git you intend to include in the *next* commit; the **repository** (the `.git` directory) is the permanent history.

```
git init                # start tracking a new repository
git status              # what's changed, what's staged, what's not
git add file.py         # stage a specific file
git add -A              # stage everything (use with care -- review status first)
git commit -m "Fix off-by-one in retry loop"
git log --oneline --graph --all  # visualize history and branches
```

4.3 Branching and merging

```
git branch feature/add-healthcheck
git checkout feature/add-healthcheck
# ...or in one step:
git checkout -b feature/add-healthcheck

# work, commit, then bring it back into main:
git checkout main
git merge feature/add-healthcheck
```

A **merge conflict** happens when Git cannot automatically combine two sets of changes to the same lines of a file, and asks a human to decide. This is normal, not a failure state — open the conflicted file, look for the «««< / ===== / »»» markers Git inserts, decide what the correct final content should be, remove the markers, then `git add` the file and complete the merge with `git commit`.

4.4 Working with remotes (GitHub, GitLab)

```
git remote add origin https://github.com/you/your-repo.git
git push -u origin main      # push and remember this as the default
                             upstream
git pull                     # fetch and merge the latest changes from
                             the remote
git fetch                    # download remote changes without merging
                             them yet
```

A **pull request** (GitHub’s term) or **merge request** (GitLab’s term) is a formal proposal to merge one branch into another, which is where code review, and in a mature pipeline, automated security scanning, happen *before* the code lands on the main branch. This is the actual mechanism that makes “shift-left security” (introduced properly in Chapter 10) real rather than a slogan: the pull request is the gate the automated scans run against.

4.5 The security-relevant Git habits

- **Never commit secrets.** A password, API key, or private key committed to Git is not truly deleted by a later commit that removes it — it remains recoverable in history. The correct response to a leaked secret is always to rotate it (change it at the source) immediately, not merely to delete the line.
- **Use a .gitignore file** to stop build artifacts, local credentials, and editor scratch files from ever being staged in the first place.
- **Sign your commits** in mature teams (GPG or SSH commit signing), so a commit’s authorship is cryptographically verifiable rather than merely a text field anyone could forge.

Case Study — poetic-musings: a real lesson from this project

While preparing the security pipeline described later in this book, a stray Emacs autosave file, `#STATUS.md#`, was discovered to have been accidentally committed to the `poetic-musings` repository months earlier, and had been sitting in history ever since as a stale, forgotten duplicate of a real status file. It was harmless in this case, but it is exactly the class of mistake —

something accidentally staged with `git add -A` and never noticed — that, with a real secret instead of a stale Markdown file, becomes a security incident. The fix was simply `git rm` and a commit explaining why, which is the same fix that applies to almost any “this should never have been committed” situation, secrets included (secrets additionally require rotation, as noted above).

Part II

Cloud Basics

Chapter 5

Cloud Computing Concepts

5.1 Concepts first, provider second

The single biggest time-waster in learning cloud computing is treating it as “learning AWS” or “learning Azure” from day one. Compute, storage, networking, and identity work the same *conceptually* on every major cloud — only the product names and exact button locations differ. Learn the concept, then learn how one specific provider (this book uses AWS, because it has the largest market share and the highest odds of matching a real job posting) implements it. That way, when a future job uses Azure or Google Cloud instead, you are translating vocabulary, not relearning computer science.

5.2 The five concepts that matter

1. **Compute** — a place your code actually runs: a virtual machine (AWS EC2), a managed container platform, or a fully managed “serverless” function that only runs when invoked (AWS Lambda).
2. **Storage** — durable places to keep data: object storage for files and blobs (AWS S3), block storage attached to a single virtual machine like a hard disk (AWS EBS), and managed databases (AWS RDS for relational data, DynamoDB for key-value data at scale).
3. **Networking** — how things talk to each other and to the internet: a Virtual Private Cloud (VPC) is your own isolated network inside the provider’s infrastructure; subnets divide that network into smaller pieces, some internet-facing (*public*), some not (*private*); security groups and network ACLs act as firewalls controlling exactly what traffic is allowed in and out.
4. **Identity and access management (IAM)** — who (a person, or a piece of software) is allowed to do what to which resource. This is arguably the single most security-critical concept in all of cloud computing, because a misconfigured permission is the most common root cause of real cloud security incidents.
5. **Regions and Availability Zones** — a *region* is a geographic area (e.g., `us-east-1`); an *Availability Zone* is one or more physically separate data centers within that region. Spreading resources across multiple Availability Zones is how you survive one data center having a bad day without your whole system going down.

5.3 The five AWS services that cover 80% of real use

If you log into the AWS console for the first time, you will see hundreds of services and feel instantly overwhelmed. Ignore nearly all of it at first. Master these five deeply before you touch anything else:

Service	What it is, in plain language
VPC	Your own private, isolated network inside AWS. Everything else you launch lives inside a VPC (or the account's default one).
EC2	Virtual machines. The classic "rent a server" compute option; still the foundation almost everything else (including Kubernetes nodes) ultimately runs on.
S3	Object storage: buckets holding files, at effectively unlimited scale, with encryption, versioning, and access controls as first-class features, not afterthoughts.
IAM	Identity and Access Management: users, roles, and policies controlling exactly what any person or program is allowed to do in your account.
ECR / EKS	Elastic Container Registry (stores container images) and Elastic Kubernetes Service (managed Kubernetes) – the on-ramp from "cloud basics" into Part 4 of this book.

5.4 IAM: the concept that will follow you through the whole book

An IAM **policy** is a document (in JSON) describing what actions are allowed or denied on which resources. An IAM **role** is an identity that something (a person, an EC2 instance, a CI pipeline) can *assume* temporarily to gain those permissions, without needing a permanent, long-lived secret key. The single most important habit to build here is **least privilege**: grant exactly the permissions a task needs, scoped to the exact resources it needs them on, and nothing more.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3::my-artifact-bucket/*"
    }
  ]
}
```

Compare that to a lazy, dangerous alternative:

```
{ "Effect": "Allow", "Action": "*", "Resource": "*" }
```

The second policy grants every possible action on every possible resource. If the credentials attached to that policy ever leak — committed to Git by accident, stolen from a compromised laptop, exfiltrated from a misconfigured CI pipeline — the damage an attacker can do is limited only by what AWS itself allows, not by anything you designed. The first policy, scoped to one action on one

specific bucket, limits the blast radius of that exact same leak to almost nothing. This single idea, applied consistently, is worth more to your security posture than almost any tool you will install.

Case Study — poetic-musings: least privilege, in real Terraform

In the `poetic-musings` platform build described throughout this book, the CI pipeline's AWS IAM role (defined in `infra/terraform/modules/aws-platform/iam.tf`) is scoped to the exact ARNs of the one ECR repository and one S3 bucket that module creates – never a wildcard `Resource: "*"`. If those pipeline credentials ever leaked, the damage would be limited to that one registry and one bucket, not the whole AWS account. This is the IAM policy shown above, applied for real.

5.5 Certifications: after proof, not instead of it

A common and expensive mistake is chasing a certification before building anything real. Certifications are valuable as a *confirmation* of knowledge you have already demonstrated through hands-on projects, not as a substitute for that hands-on work. Interviewers at a serious platform engineering team will ask you to explain a design decision or debug a scenario, not recite a multiple-choice answer. Build first. Certify second, if at all, to formalize what you already know.

Part III

Infrastructure as Code

Chapter 6

Terraform and Infrastructure as Code

6.1 The problem IaC solves

Before Infrastructure as Code was normal practice, provisioning cloud infrastructure meant a human clicking through a web console: creating a VPC here, a bucket there, an IAM role somewhere else. This has real, recurring costs:

- **Not repeatable.** Nobody remembers exactly what they clicked six months ago, so rebuilding an environment from scratch is slow and error-prone.
- **Not reviewable.** You cannot put a sequence of mouse clicks through a code review the way you can a pull request.
- **Drift.** Environments that started identical slowly diverge as different people make different manual tweaks over time, until nobody is fully sure what is actually running.
- **Not scannable.** You cannot run an automated security scanner against a person's click history the way you can against a text file.

Infrastructure as Code means describing the infrastructure you want in a text file, checking that file into Git exactly like application code, and letting a tool read that description and make the real cloud match it. Change the file, and the next run makes the cloud match the new description. You get a full history of every infrastructure change, code review on infrastructure changes, the ability to recreate an entire environment from nothing, and – critically for this book – the ability to run automated security scanners against your infrastructure *before* it is ever applied to a real account.

Terraform (by HashiCorp) is the most widely used IaC tool. You write files in HCL (HashiCorp Configuration Language) describing resources, and Terraform computes and executes the plan to create, update, or destroy real cloud resources to match what you described.

6.2 State: Terraform's memory

When Terraform creates real infrastructure, it needs to remember what it created, so a future run can tell the difference between "this already exists, leave it alone" and "this is new, create it." That memory is called **state**. By default it lives in a local file, which is dangerous the moment more than one person or pipeline needs to run Terraform concurrently – two simultaneous runs with two

different local state files can corrupt real infrastructure. The fix is a **remote backend**: state stored in a shared location (an S3 bucket is common on AWS), with a lock (often a DynamoDB table) preventing two runs from stepping on each other at once.

6.3 Providers, resources, and modules

```
# providers.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.60"
    }
  }
}
provider "aws" { region = "us-east-1" }
```

A **resource** block describes one real thing you want to exist:

```
resource "aws_s3_bucket" "artifacts" {
  bucket = "my-team-artifacts"
}

resource "aws_s3_bucket_versioning" "artifacts" {
  bucket = aws_s3_bucket.artifacts.id
  versioning_configuration { status = "Enabled" }
}
```

A **module** is a reusable, parameterized package of resources – the Terraform equivalent of a function. Instead of copy-pasting "create a VPC, an ECR repo, an S3 bucket" into every environment's configuration, you write that once as a module and call it multiple times with different inputs.

```
module "aws_platform_dev" {
  source      = "./modules/aws-platform"
  environment = "dev"
  vpc_cidr    = "10.0.0.0/16"
}

module "aws_platform_prod" {
  source      = "./modules/aws-platform"
  environment = "prod"
  vpc_cidr    = "10.1.0.0/16"
}
```

This is the core IaC lesson worth repeating until it is automatic: **do not copy-paste infrastructure definitions between environments – reuse them with different inputs.**

6.4 The Terraform workflow

```
terraform init      # download providers, set up the backend
terraform fmt       # auto-format code to a canonical style
```

```
terraform validate  # check syntax and internal consistency (no cloud
calls)
terraform plan      # show exactly what would change, without changing
anything
terraform apply     # actually make the real infrastructure match the
code
terraform destroy   # tear down everything this configuration
manages
```

`terraform plan` is the single most important habit in this whole workflow: it is a dry run that tells you precisely what will be created, changed, or destroyed, before anything actually happens. Never apply a Terraform change you have not first read the plan output for, in the same way you would never merge a pull request without reading the diff.

6.5 Hardening IaC by default

- **Encrypt at rest by default**, not as an opt-in flag – every storage resource should be encrypted unless there is a specific, documented reason not to be.
- **Block public access explicitly**, at the resource level, rather than trusting that a resource happens to default to private.
- **Scope IAM to specific resource ARNs**, as covered in Chapter 5 – this applies to Terraform-managed roles exactly as much as console-created ones.
- **Use immutable, scanned container registries** – once an image tag is pushed, prevent it from silently being overwritten, and scan every image automatically on push.
- **Tag everything consistently** (project, environment, owner) so that cost allocation, ownership tracing, and automated policy enforcement are all possible later.

6.6 Scanning infrastructure code for security issues

Just as application code gets scanned for security bugs (Chapter 10 covers this in depth), Infrastructure-as-Code should be scanned too. Tools like `tfsec` and `checkov` read your Terraform code and flag misconfigurations – a bucket without encryption, a security group open to the entire internet on a sensitive port, an IAM policy with a wildcard resource – before that configuration is ever applied to a real account.

```
tfsec infra/terraform/
checkov -d infra/terraform/
```

Case Study — poetic-musings: a real Terraform layer, built for this book

Inside `poetic-musings`, a full `infra/terraform/` tree was built exactly along these lines: a root module wiring an `aws-platform` module (VPC across two Availability Zones, a KMS-encrypted and immutable-tagged ECR repository, an SSE-KMS encrypted and versioned S3 bucket with public access explicitly blocked, a least-privilege IAM role for CI) and an `azure-platform` module (a Premium Azure Container Registry with public access disabled, a geo-redundant storage account with TLS 1.2 minimum and shared-key auth disabled), plus `environments/dev` and

`environments/prod` directories showing the same module tree reused with different `.tfvars`. Neither `terraform` nor `tfsec` was installed on the machine this was authored on, so the code was validated by hand – brace-matching across every file, and catching one real, instructive mistake along the way: an Azure provider attribute name, `enable_https_traffic_only`, that had been renamed to `https_traffic_only_enabled` in a newer provider version. That is precisely the class of small, easy-to-miss error that `terraform validate` and `tfsec` catch automatically in seconds – which is why those checks now run in the project’s CI pipeline (Chapter 9) on every push, rather than depending on a human proofreading HCL by hand forever.

Honest Gap: this Terraform has never touched a real account

Everything above has been written to be structurally correct and reviewable. None of it has been run with `terraform apply` against a live AWS or Azure account. Writing correct Infrastructure as Code and *operating* it under real conditions – real IAM federation with an identity provider, real multi-account structure, a real state-locking conflict during a busy release week – are different skills. If you are working through this book yourself, the single highest-value exercise in this entire chapter is: create a free-tier AWS account, run `terraform plan` against code like this for real, read every line of the plan output, and only then run `apply`.

Part IV

Containers and Orchestration

Chapter 7

Docker and Containers

7.1 The problem containers solve

Before containers, "it works on my machine" was a constant, expensive headache, because "my machine" had a slightly different library version, a slightly different OS, a slightly different set of installed system packages than the production server. A **container** solves this by packaging an application together with absolutely everything it needs to run – the exact language runtime version, the exact libraries, the exact OS-level dependencies – into one portable, self-contained image that behaves identically everywhere: a laptop, a CI pipeline, or a production cluster.

A container is *not* a lightweight virtual machine, even though it can feel like one. A virtual machine virtualizes an entire computer, including its own kernel. A container shares the host machine's kernel and simply isolates a process's view of the filesystem, network, and process table. This is why containers start in milliseconds where a VM takes tens of seconds, and why they are the natural building block for the fast-moving, horizontally-scaled systems this entire book is about.

7.2 Images, containers, and the Dockerfile

An **image** is a static, read-only package: a filesystem snapshot plus metadata about how to run it. A **container** is a live, running instance of that image – the relationship is the same as a class and an object in programming, or a recipe and the actual dish cooked from it. A **Dockerfile** is the recipe: a sequence of instructions describing how to build the image.

```
# ---- build stage ----
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---- runtime stage ----
FROM python:3.12-slim
RUN groupadd -g 10001 appgroup && useradd -u 10001 -g appgroup -M
    appuser
COPY --from=builder /install /usr/local
WORKDIR /app
COPY app.py .
```

```
USER 10001:10001
EXPOSE 8080
HEALTHCHECK CMD curl -f http://localhost:8080/healthz || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:8080", "app:app"]
```

7.3 Why every line above is a deliberate hardening choice

- **Multi-stage build.** The final image contains only the installed Python packages and application code – no compiler, no build tools, none of the packages only needed to *build* dependencies. Fewer tools present means fewer things an attacker who gets code execution inside the container could abuse.
- **A dedicated, unprivileged user.** By default, a container runs as `root` – the most privileged Linux user. If application code is compromised, running as `root` gives an attacker far more to work with inside the container, and a (small but real) chance of breaking further out of it. Creating `uid/gid 10001` and running as that user instead directly limits the blast radius.
- **Minimal base image.** `python:3.12-slim` instead of the full default image strips out documentation, extra utilities, and anything not strictly needed to run Python – again, less installed means less that could be vulnerable.
- **HEALTHCHECK.** Docker's own, container-level equivalent of the Kubernetes liveness probe introduced in Chapter 8 – a standard way to ask "is this container actually working," independent of whether Kubernetes is involved at all.

```
docker build -t status-service:demo .
docker run -p 8080:8080 status-service:demo
docker run --read-only --tmpfs /tmp --user 10001:10001 -p 8080:8080
    status-service:demo
docker images
docker ps
docker logs <container-id>
docker exec -it <container-id> sh
```

Case Study — poetic-musings: a real image, actually built and run

A real Flask application, `app/status-service/app.py`, was built inside `poetic-musings` specifically to have something genuine to containerize: it serves this repository's status document as HTML and JSON, with `/healthz` (a liveness check that does no I/O) and `/readyz` (a readiness check that confirms the underlying status file exists) endpoints.

The Dockerfile follows every pattern above – multi-stage, non-root `uid/gid 10001`, no unnecessary packages, pinned dependencies. It was not just written; it was actually built with `docker build`, and then `run` under the same restrictions a production Kubernetes cluster would impose:

```
docker run --read-only --tmpfs /tmp --user 10001:10001 status-service
:demo
```

The very first attempt at this failed with `FileNotFoundError: No usable temporary directory`, because `gunicorn` needs *somewhere* writable even in an otherwise fully locked-down container. That is the honest, hands-on version of "hardening" – it is a negotiation between "as locked down as possible" and "the software still has to actually run," and you find the real

boundary by testing, not by guessing or by copying a checklist you never verified. The fix – mounting a small writable `tmpfs` at exactly the one path that needs it – is exactly what the corresponding Kubernetes manifest does with an `emptyDir` volume, covered next in Chapter 8.

Chapter 8

Kubernetes

8.1 The problem Kubernetes solves

Running one container on one machine is easy. Running dozens of containers, across many machines, that need to survive crashes, scale up under load, roll out new versions without downtime, and be moved off a machine that needs maintenance – doing all of that reliably by hand does not scale past a very small team. **Kubernetes** (often "K8s") is a system that automates exactly this: you describe the *desired state* ("I want three copies of this container running, each with these resource limits, exposed on this port"), and Kubernetes continuously works to make reality match that description, healing any difference it finds automatically.

8.2 The core objects

- **Pod** – the smallest deployable unit; usually one container (sometimes a small group of tightly coupled containers) plus shared storage and network.
- **Deployment** – describes how many replicas of a Pod template should exist, and how to roll out changes to that template over time without downtime.
- **Service** – a stable network address and name for a set of Pods, since individual Pods come and go (are rescheduled, restarted, replaced) and their individual IP addresses are not stable.
- **NetworkPolicy** – a firewall for Pod-to-Pod traffic inside the cluster.
- **PodDisruptionBudget (PDB)** – a guarantee that a *voluntary* disruption (a node drain for maintenance, a cluster upgrade) will not take down more replicas at once than you have declared acceptable.

8.3 Liveness vs. readiness

This distinction is asked about in interviews constantly, and is worth being able to explain crisply. A **liveness probe** answers "is this container still alive?" – if it fails, Kubernetes assumes the container is stuck or dead and restarts it. A **readiness probe** answers a different question: "is this container ready to serve real traffic *right now*?" – a container can be alive but not ready (for example, still

loading data on startup), and Kubernetes will withhold traffic from it without restarting it. Liveness failing means *restart me*. Readiness failing means *don't send me traffic yet, but leave me running*.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: status-service
spec:
  replicas: 3
  selector:
    matchLabels: { app: status-service }
  template:
    metadata:
      labels: { app: status-service }
    spec:
      securityContext:
        runAsNonRoot: true
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: status-service
          image: registry.example.com/status-service:v1.0.0
          ports: [{ containerPort: 8080 }]
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "250m", memory: "256Mi" }
          securityContext:
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
            capabilities: { drop: ["ALL"] }
          livenessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 10
          readinessProbe:
            httpGet: { path: /readyz, port: 8080 }
            initialDelaySeconds: 2
            periodSeconds: 5
          volumeMounts:
            - { name: tmp, mountPath: /tmp }
      volumes:
        - name: tmp
          emptyDir: {}
```

8.4 The restricted Pod Security Standard, explained line by line

Every `securityContext` field above corresponds to a specific, named threat:

- `runAsNonRoot: true` – a second, independent enforcement of the same non-root principle from the Dockerfile in Chapter 7. Defense in depth means never trusting a single layer to get a control right.
- `readOnlyRootFilesystem: true` – the container's filesystem cannot be written to at all, except for volumes explicitly mounted as writable (here, the `emptyDir` at `/tmp`). An attacker

with code execution inside the container has (almost) nowhere to write a malicious file.

- `capabilities.drop`: `["ALL"]` – Linux capabilities are fine-grained permissions beyond simple root/non-root (binding low-numbered ports, changing file ownership, and so on). Dropping all of them removes every one of these extra powers unless a specific one is explicitly re-added – the same least-privilege instinct as the IAM policy in Chapter 5, applied at the kernel-capability layer.
- `allowPrivilegeEscalation`: `false` – prevents a process from gaining more privileges than it started with, for example via a `setuid` binary.
- `seccompProfile`: `RuntimeDefault` – restricts which raw Linux system calls the container’s processes are allowed to make at all, to a sane, restrictive default set instead of everything the kernel supports.

Together, these implement what Kubernetes calls the **restricted** Pod Security Standard – the strictest of its three built-in profiles (*privileged*, then *baseline*, then *restricted*). Being able to name this standard and explain each control’s purpose, unprompted, is a genuinely strong signal in an interview.

8.5 Network policy: default-deny

By default, every Pod in a Kubernetes cluster can talk to every other Pod, on any port. A `NetworkPolicy` turns that off. The correct pattern is **default-deny, then explicit allow**:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny-all }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-status-service-ingress }
spec:
  podSelector: { matchLabels: { app: status-service } }
  policyTypes: [Ingress]
  ingress:
    - ports: [{ protocol: TCP, port: 8080 }]
```

8.6 Kustomize: one base, many environments

Exactly as Terraform modules avoid copy-pasting infrastructure between environments, **Kustomize** (built into `kubectl`) lets a `base/` set of manifests be patched per environment without duplicating the whole file.

```
kubectl apply -k k8s/overlays/dev
kubectl apply -k k8s/overlays/prod
kustomize build k8s/overlays/prod
```

Case Study — poetic-musings: a real, tested Kubernetes layer

The `k8s/` directory in `poetic-musings` deploys the same status-service image from Chapter 7: a `base/` with a Deployment implementing every control described above, a Service, three separate `NetworkPolicy` objects (default-deny, explicit ingress on 8080, explicit DNS egress), and a `PodDisruptionBudget` with `minAvailable: 1`, paired with a 3-replica `overlays/prod` Kustomize overlay (`overlays/dev` runs 1 replica).

No real Kubernetes cluster was available while building this, so the manifests were validated two other ways: every YAML file was parsed to confirm it is syntactically valid, and `kustomize build` was run against both overlays to confirm they render into complete, valid Kubernetes objects with no errors – which caught one real issue, a deprecated `commonLabels` Kustomize field that needed to become `labels` in newer Kustomize versions. That is the same lesson as the Terraform provider-attribute mistake in Chapter 6: a tool designed to catch exactly this class of drift caught it, once it was actually run.

Honest Gap: this has never scheduled a real Pod

Writing a correct manifest and watching Kubernetes actually enforce it, under real load, across real node failures, are different skills, and only the first is demonstrated here. The highest-value next exercise: install `kind` (Kubernetes-in-Docker, free, no cloud account required) and run `kubectl apply -k k8s/overlays/dev` against a real, if small, cluster. Watch the Pod actually schedule. Then deliberately break a security setting – remove `readOnlyRootFilesystem`, for instance – and observe exactly what changes, so the value of each control stops being theoretical.

Part V

CI/CD and Security Automation

Chapter 9

CI/CD: The Backbone of DevOps

9.1 Why this is the highest-leverage phase

Every earlier phase in this book – Terraform provisioning infrastructure, Docker building images, Kubernetes running them – becomes genuinely valuable only once it is wired together by automation that runs on every code change, without a human remembering to trigger each step by hand. CI/CD is that wiring, which is why the road map this book follows treats it as the phase deserving the most sustained attention.

Continuous Integration (CI) is the practice of automatically building and testing every change, so problems are caught within minutes of being introduced. **Continuous Delivery** extends this to automatically leaving every passing change ready to deploy, with a human clicking a final button. **Continuous Deployment** goes one step further and deploys automatically, with no human step at all. Learn the concepts once; the specific tool (Jenkins, GitHub Actions, GitLab CI, Azure DevOps) is a dialect, not a different language.

9.2 The anatomy of a pipeline

A pipeline is a sequence of **stages**, each containing one or more **jobs**, triggered by an event – typically a push or a pull/merge request. A well-designed pipeline is staged for **fail-fast**: cheap, fast checks run before expensive, slow ones, so a broken build is caught in seconds, not after a ten-minute container build that was never going to matter.

```
# The rough shape of a well-staged pipeline, tool-agnostic:
# 1. lint (seconds)
# 2. unit tests (tens of seconds)
# 3. secrets scan (seconds, full git history)
# 4. SAST (tens of seconds)
# 5. build the image (a minute or two)
# 6. container scan (a minute or two)
# 7. IaC / K8s manifest scans
# 8. deploy (gated on everything above passing)
```


Chapter 10

Shifting Security Left

10.1 What “shift left” actually means

Historically, security review happened at the far right end of a project’s timeline: a separate team audited the software right before launch, often too late to fix anything without expensive rework, and often skipped entirely under deadline pressure. **Shifting left** means moving that review as early as possible – ideally into the same automated pipeline that runs on every single commit – so a mistake is caught in a thirty-second scan the moment it is introduced, rather than discovered by an attacker (or an auditor) months later. This is the defining idea of DevSecOps: security is not a gate at the end, it is a property enforced continuously, automatically, throughout.

10.2 The layers of automated security scanning

A mature pipeline stacks several different scanners, because no single tool catches everything – each is aimed at a different layer of the system, an approach usually called **defense in depth**.

10.2.1 Secrets detection

Scans for anything that looks like a password, API key, or private key committed to the repository – critically, across the *entire git history*, not merely the current diff, because a secret committed and later “removed” in a subsequent commit is still retrievable from history by anyone with read access, until it is rotated.

```
# gitleaks, run against full history
- name: Secrets scan
  run: gitleaks detect --source . --log-opts="--all" --exit-code 1
```

10.2.2 SAST – Static Application Security Testing

Analyzes source code for security-relevant patterns *without running it*: building a shell command by concatenating untrusted input (command injection risk), a hardcoded credential, a known-weak cryptographic function. This differs from a linter, which cares about style and correctness; a SAST tool specifically targets security-relevant code shapes.

```
bandit -r app -ll -f sarif -o bandit.sarif
```

10.2.3 SBOM and dependency scanning (SCA)

A **Software Bill of Materials (SBOM)** is a complete, machine-readable inventory of every library your software depends on and its exact version – an ingredient list. **Dependency scanning** (or Software Composition Analysis, SCA) checks that inventory against a database of known, publicly disclosed vulnerabilities (CVEs – Common Vulnerabilities and Exposures). The SBOM answers "what's in this artifact"; the dependency scan answers "does any of it have a known flaw."

```
syft . -o cyclonedx-json=sbom.cdx.json
pip-audit -r requirements.txt
```

10.2.4 Container image scanning

Scans the *built* container image – every layer, including the base OS image, not just your application's direct dependencies. A vulnerability can exist in a system library baked into the base image that your dependency file never even mentions.

```
trivy image --severity HIGH,CRITICAL --exit-code 1 --ignore-unfixed my-app:ci
```

10.2.5 IaC and Kubernetes manifest scanning

The same category of check as SAST, aimed at infrastructure definitions and deployment manifests instead of application code, as introduced in Chapter 6 and Chapter 8.

```
tfsec infra/terraform/
kube-linter lint k8s/
```

10.3 Pipeline design habits that mark seniority

- **Least-privilege pipeline permissions.** Each job in the pipeline should declare exactly what it needs (read-only by default; write access to security-scanning results only on the specific jobs that upload them) – the CI equivalent of the IAM scoping from Chapter 5, applied to the pipeline's own credentials.
- **Pin third-party actions/plugins to a specific version,** never to a floating branch like `@main`. An unpinned CI dependency is a supply-chain risk: if that dependency's own repository is ever compromised, every pipeline pointing at its moving branch picks up the malicious change automatically and silently.
- **Centralize findings.** Several scanners can emit a standard format called **SARIF** (Static Analysis Results Interchange Format); uploading all of them to one dashboard means findings live in one place instead of being scattered across seven different job logs nobody reads.
- **Design for incremental rollout.** In a real organization, a platform pipeline is often adopted by different teams at different times. A well-designed pipeline checks whether the file or directory it needs even exists yet, and skips cleanly with a note rather than hard-failing, for teams that have not migrated onto a given piece yet.
- **Re-run on a schedule, not only on push.** A dependency that was safe last week can have a newly disclosed CVE this week even though no code changed. Security scanning is not a

one-time gate at commit time – vulnerability databases update continuously, so the scan needs to repeat on a schedule too.

Case Study — poetic-musings: a full security pipeline, built and validated

Two workflow files were built for `poetic-musings` in GitHub Actions: `ci.yml` (lint with `ruff`, run any unit tests) and `security.yml`, which runs on every push and pull request *and* on a weekly cron, implementing every layer above: `gitleaks` (full-history secrets scan), `bandit` (SAST, SARIF uploaded to GitHub code scanning), `syft` + `pip-audit` (SBOM and dependency scan), `trivy` (container image scan, gated on HIGH/CRITICAL findings), `tfsec` (IaC scan of the Terraform from Chapter 6), and `kube-linter` (manifest scan of the Kubernetes YAML from Chapter 8).

Every job that touches a path another part of the platform owns (the Dockerfile, the Terraform tree, the Kubernetes manifests) checks that the path actually exists first, and skips cleanly with a step-summary note if it does not – so the pipeline was safe to land before every other piece of the platform existed yet, and self-activated as each one landed, exactly the incremental-rollout pattern described above. Permissions were scoped per job (read-only by default, write access to security findings only where a job actually uploads SARIF output), and every third-party action was pinned to a specific version.

The same pipeline was then reproduced, stage for stage, as `.gitlab-ci.yml` using GitLab CI’s own syntax (`stages:`, `rules:` `exists:` standing in for the path-existence checks, GitLab’s built-in `artifacts:` blocks) – specifically because a real job posting for this kind of role is just as likely to name GitLab as GitHub, and the fastest way to confirm the underlying concepts really did transfer, rather than merely memorizing one tool’s YAML dialect, is to rebuild the same pipeline in a second one.

Honest Gap: none of these pipelines have executed against a real runner

Both pipeline files were validated for correct YAML syntax and reviewed by hand for structure, permissions scoping, and action pinning – but neither has actually executed end to end on a real GitHub Actions or GitLab CI runner, which is the only way to prove a pipeline truly works rather than merely reads correctly. Pushing this code to a real GitHub repository and a real free-tier GitLab project, and watching both pipelines run for real, is the concrete next step – and it is genuinely inexpensive: GitHub Actions and GitLab.com both include a meaningful amount of free CI minutes for exactly this kind of practice.

Part VI

Observability

Chapter 11

Prometheus and Grafana

11.1 Why observability is the natural last step

Everything built so far – infrastructure, containers, deployments, pipelines – can, and eventually will, misbehave in production. **Observability** is the practice of being able to answer "what is my system actually doing right now, and why," without having to guess or manually SSH into a machine to poke around. It is deliberately the last phase in this road map, because it is most useful once you already have real, running systems (from Parts 4 and 5) worth observing.

11.2 Metrics, the core concept

A **metric** is a numeric measurement of your system over time – requests per second, error rate, CPU usage, memory consumption, queue depth. **Prometheus** is the dominant open-source system for collecting and storing these metrics: it periodically *scrapes* (pulls) metrics from every service you configure it to watch, over HTTP, and stores them as a time series.

```
# A minimal application exposes metrics at a well-known path:
GET /metrics
# http_requests_total{method="GET",status="200"} 1523
# http_request_duration_seconds_bucket{le="0.1"} 1400
```

Prometheus's own query language, **PromQL**, lets you ask questions of that stored data:

```
rate(http_requests_total{status="500"}[5m])    # error rate over the last
5 minutes
histogram_quantile(0.95, http_request_duration_seconds_bucket) # p95
latency
```

Grafana is the visualization layer on top: it queries Prometheus (and other data sources) and renders the results as dashboards – graphs, gauges, alert states – that a human (or an on-call engineer at 2am) can actually read at a glance, rather than staring at raw numbers.

11.3 Alerting: the point of all of it

Metrics and dashboards are only useful if someone is looking at them at the right moment, which is why the practical endpoint of observability is **alerting**: a rule that says "if this condition holds for this long, notify a human," typically via **Alertmanager** (Prometheus's companion alerting component) routing to email, Slack, or a paging system.

```
groups:
- name: status-service
  rules:
  - alert: HighErrorRate
    expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
    for: 10m
    labels: { severity: warning }
    annotations:
      summary: "Error rate above 5% for 10 minutes"
```

The security connection

Observability is not only about performance. The same metrics pipeline that alerts on elevated error rates can alert on a spike in failed authentication attempts, an unusual volume of requests from one source, or a container restarting far more often than normal – all of which can be early signals of an attack in progress, not merely a performance problem. A platform engineer who treats observability purely as a "site reliability" concern, separate from security, is missing half of its real value.

Honest Gap: not yet built in this project

No Prometheus/Grafana stack has been built inside *poetic-musings* as of this writing – the status-service application from Chapter 7 does not yet expose a `/metrics` endpoint, and no Prometheus scrape configuration or Grafana dashboard exists for it. This is a deliberately honest, named gap rather than a claimed accomplishment. The natural next exercise: add a metrics library (for Flask, commonly `prometheus_client`) to the status-service, expose request-count and latency metrics at `/metrics`, run a local Prometheus and Grafana via Docker Compose, and build one real dashboard showing that application's live traffic.

Part VII

The Enterprise Layer: Windows, Identity, and Hardening at Scale

Chapter 12

Active Directory, GPOs, and Endpoint Management

12.1 Why this belongs in a cloud-native book

Everything in Parts 2 through 6 of this book is cloud-native and Linux-centric, which mirrors most of modern platform engineering. But the job this book is aimed at also explicitly asks for Windows Server administration, Active Directory, Group Policy Objects, and endpoint management tooling (MECM/SCCM, Intune) – because most real enterprises are hybrid: cloud-native workloads running alongside a Windows-based corporate identity and endpoint fleet that predates (and will likely outlive) any single cloud migration. A platform engineer is expected to be credible across that whole boundary, not just the cloud-native half.

12.2 Active Directory: the identity backbone

Active Directory (AD) is Microsoft's directory service: a database of every user, computer, and group in an organization, along with the relationships and permissions between them, organized into a hierarchical structure of *domains*, *organizational units (OUs)*, and *forests* (a forest is a collection of one or more domains that trust each other). When an employee logs into their laptop, or a service account authenticates to an internal application, AD is very often the system answering "is this really who they claim to be, and what are they allowed to do."

Azure AD Connect (now more often referred to under the Microsoft Entra ID branding) is the bridge between an on-premises AD and its cloud equivalent: it synchronizes users and groups from the on-prem directory into the cloud directory, most commonly via *password hash synchronization* (a one-way hash of each user's password is synced, so the cloud can authenticate them without storing their plaintext password) or *pass-through authentication* (the cloud actually validates the credential against the on-prem domain controller in real time), often paired with *seamless single sign-on* so a user on a domain-joined machine is not prompted to log in again for cloud resources they are already authenticated for on the corporate network.

12.3 Group Policy Objects: configuration as (a kind of) code

A **Group Policy Object (GPO)** is a declared set of configuration settings – password complexity requirements, which applications are allowed to run, firewall rules, drive mappings, screen-lock timeouts – that AD pushes out automatically to every computer or user account within its scope. Conceptually, this is the same idea as Infrastructure as Code from Chapter 6, applied to endpoint configuration instead of cloud resources: a declared desired state, enforced automatically across a whole fleet, instead of a human configuring each machine by hand one at a time. If you already understand why Terraform beats manual console clicks, you already understand why GPOs beat a technician walking desk to desk configuring each laptop individually.

12.4 MECM/SCCM and Intune: the enforcement arm

MECM (Microsoft Endpoint Configuration Manager, the modern name for what was long known as **SCCM**, System Center Configuration Manager) is the traditional tool for managing on-premises, domain-joined Windows devices at scale: pushing software installs and updates, enforcing configuration baselines, and reporting on device compliance across potentially tens of thousands of machines. **Intune** is Microsoft's cloud-native counterpart, managing devices – including mobile devices – without requiring them to be on the corporate network or domain-joined at all, which has made it increasingly central as workforces became more remote and hybrid. Many enterprises run both, side by side, in a "co-management" configuration during a multi-year migration from one to the other.

These tools are the endpoint-level enforcement arm of exactly the same hardening philosophy running through every other chapter of this book: a GPO or an Intune configuration profile that disables a legacy, insecure protocol on every laptop in the company is the same instinct as the S3 public-access block in Chapter 5, or the `readOnlyRootFilesystem` setting in Chapter 8 – make the safe behavior the automatically-enforced default, everywhere, rather than a policy document nobody reads.

Honest Gap: no real Windows domain exists in this project

None of this chapter's material has a natural home inside a Linux-based git repository the way Terraform or Kubernetes manifests do – it genuinely requires real (or lab) Windows Server infrastructure to practice hands-on. The single highest-value exercise for closing this specific gap, and arguably the highest-value exercise in this entire book, given how much of the target job description it covers: build a small home lab. Two or three virtual machines (VirtualBox, Hyper-V, or a low-cost cloud VM) are enough – one running Windows Server with the Active Directory Domain Services role installed and a new forest promoted, a second machine joined to that domain as a member. From there, practice creating organizational units, users, and groups; author one real GPO (for example, enforcing a password-complexity policy or a screen-lock timeout) and confirm on the joined machine that it actually applied. Microsoft also offers free trial access to Intune specifically for this kind of self-directed study, with no production environment required.

Chapter 13

CIS Benchmarks and Formal Hardening

13.1 What a CIS Benchmark actually is

The **Center for Internet Security (CIS)** publishes detailed, versioned, freely available hardening checklists – called **CIS Benchmarks** – for specific systems: one for Windows Server, one for a given Linux distribution, one for Kubernetes, one for each major cloud provider, and more. Each benchmark is a long, specific list of individual settings, each with a stated rationale and often two *levels*: Level 1 (basic hardening, broadly safe to apply almost everywhere) and Level 2 (more restrictive, intended for genuinely high-security environments where the operational trade-offs are acceptable).

13.2 From "aligned with" to "scored against"

Every hardening decision made throughout this book – encryption at rest by default, public access blocked explicitly, least-privilege IAM, the restricted Kubernetes Pod Security Standard, non-root containers, default-deny networking – is **CIS-aligned in spirit**: each one reflects the same underlying philosophy the actual CIS Benchmarks codify. That is meaningfully different from having been formally **scored** against an actual benchmark using a real scanning tool run against a live account or cluster, which produces a compliance percentage and an itemized list of exactly which specific benchmark controls pass and fail.

```
# kube-bench: runs the CIS Kubernetes Benchmark against a live cluster
kube-bench run --targets master,node

# AWS: enable a CIS-aligned AWS Config conformance pack against a real
      account
# Azure: enable the CIS benchmark initiative under Azure Policy
```

Honest Gap: no formal, scored compliance report exists yet

Nothing built for this book has been run through a formal CIS Benchmark scanner against a live account or cluster – there is no live account or cluster to score. Once the Kubernetes gap from Chapter 8 and the Terraform gap from Chapter 6 are closed (a real **kind** cluster, a real AWS free-tier account), running **kube-bench** against that cluster and enabling a CIS conformance

pack in AWS Config against that account are natural, low-cost follow-on exercises, and would produce the first genuinely *scored* artifact in this whole learning path.

Chapter 14

Managed Development Environments: Coder

The job this book is built around specifically names prior experience with **Coder**, a platform for provisioning and managing remote, browser-accessible development environments – the idea being that instead of every engineer configuring their own laptop by hand, a platform team defines standard, reproducible *workspace templates* (themselves often just Terraform, tying directly back to Chapter 6) that provision a consistent, secure development environment for anyone on the team in minutes, running on the organization’s own infrastructure rather than an engineer’s personal machine.

Coder itself is most commonly deployed onto Kubernetes via a Helm chart, which means the Kubernetes hardening patterns from Chapter 8 – NetworkPolicy, the restricted Pod Security Standard, resource limits, PodDisruptionBudgets – are directly reusable as the security baseline for the workspaces Coder provisions, not a separate skill to learn from scratch.

Honest Gap: not built in this project

No Coder deployment exists inside *poetic-musings*. Once the Kubernetes gap from Chapter 8 is closed with a real `kind` or cloud cluster, installing Coder’s Helm chart onto that same cluster and standing up one real workspace template is a direct, concrete next exercise, and one that would let this specific named requirement move from "understood conceptually" to "operated hands-on."

Part VIII

Putting It All Together

Chapter 15

Avoiding the Isolated-Learning Trap, for Real

15.1 The mistake, restated

Chapter 1 named the central failure mode this book is built to avoid: learning each tool in its own bubble, never combining them, ending up with vocabulary instead of the connective tissue that actually makes an engineer valuable. It is worth restating now, at the end, because it is easy to read eleven chapters of concepts and case studies and still fall into exactly this trap in your own practice, if you treat this book itself as one more course to finish rather than a structure to build *inside of*.

15.2 The discipline that fixes it

1. **One project, continuously expanded.** Every case study in this book built on the *same* small platform, in the same repository, rather than starting a fresh toy example for each tool. Do the same in your own practice: pick one project, and every new tool you learn gets applied to *it*, not to a disconnected new example.
2. **Build before you certify.** Certifications formalize knowledge you have already demonstrated hands-on; they are a poor substitute for that hands-on work, and interviewers can tell the difference immediately.
3. **Notice the feeling of a knowledge gap, and go close it.** That feeling of insecurity reading a job description, or freezing on an interview question, is diagnostic information, not a verdict on your ability – it is telling you precisely where to spend your next study session.
4. **Say what you have not proven, plainly.** Every "Honest Gap" box in this book exists because naming a limitation precisely is a mark of seniority, not weakness. A senior engineer who says "I have written this Terraform correctly, but I have not operated it against a live multi-account AWS org under real IAM federation" is more credible, not less, than one who implies they have done everything.

Chapter 16

Consolidated Gap List and Next Exercises

16.1 Every honest gap named in this book, in one place

Gap	Concrete next step
Terraform never <code>apply</code> 'd to a real account	Create a free-tier AWS account; run <code>terraform plan</code> , read it fully, then <code>apply</code> against real (small, free-tier) resources.
Kubernetes manifests never scheduled on a real cluster	Install <code>kind</code> ; run <code>kubect1 apply -k k8s/overlays/dev</code> ; deliberately break one security setting and observe the effect.
CI/CD pipelines never executed on a real runner	Push to a real GitHub repository and a free GitLab.com project; watch both pipelines run end to end; fix whatever actually fails (something will).
No observability stack built	Add <code>/metrics</code> to the status-service; run Prometheus and Grafana locally via Docker Compose; build one real dashboard.
No Windows AD / GPO / MECM / Intune hands-on	Build a two-to-three VM home lab: a domain controller, a joined member machine, one real GPO, confirmed applied. Trial Intune separately.
No formal, scored CIS Benchmark compliance report	Once a real cluster and account exist (above), run <code>kube-bench</code> and enable a CIS conformance pack in AWS Config.
No Coder deployment	Once a real cluster exists, install Coder's Helm chart onto it and stand up one workspace template.

16.2 A study cadence

Following the road map's own time estimates, a realistic, sustained cadence closing the gaps above, on top of everything already built for this book, is roughly ten to twelve additional weeks of deliberate, hands-on practice – not years, and not full-time, but consistent. Protect a fixed block of time for it every week, treat the gap list above as the backlog, and resist the pull to restart from a fresh tutorial instead of extending the one real project you already have. That, more than any

single technology in this book, is the actual skill this whole field rewards.

Glossary

AD (Active Directory)

Microsoft's directory service for identity, authentication, and authorization across an organization's users, computers, and groups.

CI/CD

Continuous Integration / Continuous Delivery (or Deployment): automatically building, testing, and (for Deployment) releasing code on every change.

CIS Benchmark

An industry-standard, detailed hardening checklist for a specific system, published by the Center for Internet Security.

Container

A packaged, portable unit of software including everything it needs to run, so it behaves identically across machines.

CVE

Common Vulnerabilities and Exposures: the standard public naming/numbering system for disclosed security vulnerabilities.

DevSecOps

DevOps with security scanning built directly into the automated pipeline, rather than handled as a separate, later review.

Docker

The most common tool for building and running containers.

GPO (Group Policy Object)

A declared configuration/security setting that Active Directory pushes automatically to computers or users in its scope.

IaC (Infrastructure as Code)

Describing infrastructure in version-controlled text files instead of manual cloud-console clicks, so it is repeatable, reviewable, and recreatable.

IAM (Identity and Access Management)

The system for controlling who or what can do what to which resource.

IDP (Internal Developer Platform)

The shared infrastructure, pipelines, and tooling a platform team builds for every other team to build on.

Kubernetes (K8s)

A system for running and automatically managing many containers across many machines.

Least privilege

Granting exactly the access needed and no more, so a compromised credential or component causes the smallest possible damage.

MECM/SCCM

Microsoft Endpoint Configuration Manager (formerly System Center Configuration Manager): tooling for managing on-premises Windows devices at scale.

Module (Terraform)

A reusable, parameterized package of infrastructure resources.

PromQL

The query language used to ask questions of data stored in Prometheus.

Pod Security Standards

Kubernetes' three built-in security profiles for Pods: privileged, baseline, and restricted (strictest).

SARIF

Static Analysis Results Interchange Format: a standard format for security scan results, so different tools' findings can be viewed in one place.

SAST

Static Application Security Testing: analyzing source code for security-relevant patterns without running it.

SBOM

Software Bill of Materials: a complete, machine-readable inventory of every dependency a piece of software uses.

Shift left

Moving security checks earlier in the development timeline (into the automated pipeline, at commit time) instead of as a late, pre-launch review.

State (Terraform)

Terraform's record of what infrastructure it has already created, used to compute what needs to change on the next run.

Terraform

The most widely used Infrastructure-as-Code tool, using the HCL language.

About This Book

This book was written as a hands-on companion to a real, working platform built inside the `poetic-musings` git repository – an FPGA/hardware/music hobby project that had no cloud or container footprint before this material was built into it, specifically to give every concept in this book something real to point at. The relevant paths in that repository are: `infra/terraform/`, `app/status-service/`, `k8s/`, `.github/workflows/`, `.gitlab-ci.yml`, `SECURITY.md`, and the companion documents `DEFSECOPS.md` and `DEVSECOPS.md`.